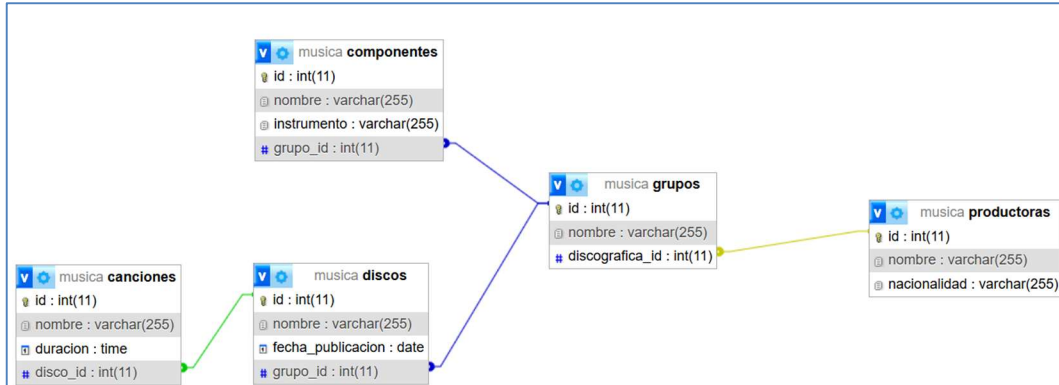


Relación de ejercicios II – BADA – Ud.6 PL-SQL

A partir de la base de datos *musica.sql* que puedes observar en la imagen, desarrolla las siguientes peticiones en PLSQL.



CURSORES

1. Listar canciones de cada disco

Crea un cursor que recorra todos los discos y, para cada uno, muestre todas sus canciones con su duración.

```

DECLARE
  CURSOR c_discos IS
    SELECT id, nombre
    FROM discos;

  v_disco_id discos.id%TYPE;
  v_disco_nombre discos.nombre%TYPE;

  CURSOR c_canciones (p_disco_id discos.id%TYPE) IS
    SELECT nombre, duracion
    FROM canciones
    WHERE disco_id = p_disco_id;

  v_cancion_nombre canciones.nombre%TYPE;
  v_cancion_duracion canciones.duracion%TYPE;

BEGIN
  OPEN c_discos;
  LOOP
    FETCH c_discos INTO v_disco_id, v_disco_nombre;
    EXIT WHEN c_discos%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Disco: ' || v_disco_nombre);

    OPEN c_canciones(v_disco_id);
    LOOP
      FETCH c_canciones INTO v_cancion_nombre, v_cancion_duracion;
      EXIT WHEN c_canciones%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(' - ' || v_cancion_nombre || ' (' || v_cancion_duracion || ')');
    END LOOP;
    CLOSE c_canciones;
  END LOOP;

```

```
END LOOP;  
CLOSE c_discos;  
END;  
/
```

Sin hacer uso de fetch:

```
DECLARE  
CURSOR c_discos IS  
  SELECT id, nombre  
  FROM discos;  
  
CURSOR c_canciones (p_disco_id discos.id%TYPE) IS  
  SELECT nombre, duracion  
  FROM canciones  
  WHERE disco_id = p_disco_id;  
  
BEGIN  
  FOR rec_disco IN c_discos LOOP  
    DBMS_OUTPUT.PUT_LINE('Disco: ' || rec_disco.nombre);  
  
    FOR rec_cancion IN c_canciones(rec_disco.id) LOOP  
      DBMS_OUTPUT.PUT_LINE(' - ' || rec_cancion.nombre || ' (' || rec_cancion.duracion || ')');  
    END LOOP;  
  END LOOP;  
END;  
/
```

2. Listar integrantes de cada grupo y su instrumento

Implementa un cursor que recorra los grupos musicales y, para cada grupo, liste a sus integrantes junto con el instrumento que tocan.

```
DECLARE  
  -- Cursor para recorrer los grupos musicales  
  CURSOR c_grupos IS  
    SELECT id, nombre  
    FROM grupos;  
  
  -- Variables para almacenar los datos del grupo  
  v_grupo_id grupos.id%TYPE;  
  v_grupo_nombre grupos.nombre%TYPE;  
  
  -- Cursor para recorrer los componentes de un grupo  
  CURSOR c_componentes (p_grupo_id grupos.id%TYPE) IS  
    SELECT nombre, instrumento  
    FROM componentes  
    WHERE grupo_id = p_grupo_id;  
  
  -- Variables para almacenar los datos del componente  
  v_componente_nombre componentes.nombre%TYPE;  
  v_componente_instrumento componentes.instrumento%TYPE;  
  
BEGIN  
  -- Abrir el cursor de grupos  
  OPEN c_grupos;
```

```
-- Loop principal para recorrer los grupos
LOOP
  -- Obtener los datos del grupo
  FETCH c_grupos INTO v_grupo_id, v_grupo_nombre;

  -- Salir del loop si no hay más grupos
  EXIT WHEN c_grupos%NOTFOUND;

  -- Imprimir el nombre del grupo
  DBMS_OUTPUT.PUT_LINE('Grupo: ' || v_grupo_nombre);

  -- Abrir el cursor de componentes para el grupo actual
  OPEN c_componentes(v_grupo_id);

  -- Loop anidado para recorrer los componentes del grupo
  LOOP
    -- Obtener los datos del componente
    FETCH c_componentes INTO v_componente_nombre, v_componente_instrumento;

    -- Salir del loop si no hay más componentes
    EXIT WHEN c_componentes%NOTFOUND;

    -- Imprimir el nombre del componente y el instrumento
    DBMS_OUTPUT.PUT_LINE('      - ' || v_componente_nombre || ' (' ||
v_componente_instrumento || ');');
  END LOOP;

  -- Cerrar el cursor de componentes
  CLOSE c_componentes;
END LOOP;

-- Cerrar el cursor de grupos
CLOSE c_grupos;
END;
/
```

Sin usar fetch:

```
DECLARE
  CURSOR c_grupos IS
    SELECT id, nombre
    FROM grupos;

  CURSOR c_componentes (p_grupo_id grupos.id%TYPE) IS
    SELECT nombre, instrumento
    FROM componentes
    WHERE grupo_id = p_grupo_id;

BEGIN
  FOR rec_grupo IN c_grupos LOOP
    DBMS_OUTPUT.PUT_LINE('Grupo: ' || rec_grupo.nombre);

    FOR rec_componente IN c_componentes(rec_grupo.id) LOOP
      DBMS_OUTPUT.PUT_LINE('      - ' || rec_componente.nombre || ' (' ||
rec_componente.instrumento || ');');
    END LOOP;
  END LOOP;
```

```
END LOOP;  
END;  
/
```

3. Actualizar la duración de canciones cortas

Utiliza un cursor para identificar canciones cuya duración sea menor a 2 minutos y actualízalas a 2 minutos exactos.

```
DECLARE  
CURSOR c_canciones IS  
  SELECT id, duracion  
  FROM canciones  
  WHERE duracion LIKE '01%' OR duracion LIKE '00%'  
  ORDER BY duracion;  
  
BEGIN  
  FOR registro IN c_canciones LOOP  
    DBMS_OUTPUT.PUT_LINE('Id_Canción: ' || registro.id || ' Duración: ' || registro.duracion);  
    UPDATE canciones SET duracion = '02:00:00' WHERE id=registro.id;  
  END LOOP;  
END;  
/
```

Usando fetch:

```
DECLARE  
CURSOR c_canciones IS  
  SELECT id, duracion  
  FROM canciones  
  WHERE duracion LIKE '01%' OR duracion LIKE '00%'  
  ORDER BY duracion;  
  
v_cancion_id canciones.id%TYPE;  
v_cancion_duracion canciones.duracion%TYPE;  
  
BEGIN  
  OPEN c_canciones;  
  LOOP  
    FETCH c_canciones INTO v_cancion_id, v_cancion_duracion;  
    EXIT WHEN c_canciones%NOTFOUND;  
  
    DBMS_OUTPUT.PUT_LINE('Id_Canción: ' || v_cancion_id || ' Duración: ' ||  
v_cancion_duracion);  
    UPDATE canciones SET duracion = '02:00:00' WHERE id = v_cancion_id;  
  END LOOP;  
  CLOSE c_canciones;  
  COMMIT; -- Importante: guarda los cambios  
END;  
/
```

4. Mostrar grupos con su discografía

Diseña un cursor que recorra los grupos y muestre el nombre del grupo junto con el nombre de su discografía.

```
DECLARE  
CURSOR c_grupos_discograficas IS
```

```
SELECT g.nombre AS nombre_grupo, p.nombre AS nombre_discografica
FROM grupos g
JOIN productoras p ON g.discografica_id = p.id;

v_nombre_grupo grupos.nombre%TYPE;
v_nombre_discografica productoras.nombre%TYPE;

BEGIN
OPEN c_grupos_discograficas;
LOOP
FETCH c_grupos_discograficas INTO v_nombre_grupo, v_nombre_discografica;
EXIT WHEN c_grupos_discograficas%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('Grupo: ' || v_nombre_grupo || ', Discográfica: ' ||
v_nombre_discografica);
END LOOP;
CLOSE c_grupos_discograficas;
END;
/
```

Sin usar Fetch:

```
DECLARE
CURSOR c_grupos_discograficas IS
SELECT g.nombre AS nombre_grupo, p.nombre AS nombre_discografica
FROM grupos g
JOIN productoras p ON g.discografica_id = p.id;

BEGIN
FOR rec_grupo_discografica IN c_grupos_discograficas LOOP
DBMS_OUTPUT.PUT_LINE('Grupo: ' || rec_grupo_discografica.nombre_grupo || ', Discográfica:
' || rec_grupo_discografica.nombre_discografica);
END LOOP;
END;
/
```

5. Total de canciones por disco

Crea un cursor que recorra los discos y muestre cuántas canciones tiene cada uno.

```
DECLARE
CURSOR c_discos_canciones IS
SELECT d.nombre AS nombre_disco, COUNT(c.id) AS numero_canciones
FROM discos d
LEFT JOIN canciones c ON d.id = c.disco_id
GROUP BY d.nombre;

v_nombre_disco discos.nombre%TYPE;
v_numero_canciones NUMBER;

BEGIN
OPEN c_discos_canciones;
LOOP
FETCH c_discos_canciones INTO v_nombre_disco, v_numero_canciones;
EXIT WHEN c_discos_canciones%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('Disco: ' || v_nombre_disco || ', Canciones: ' || v_numero_canciones);
```

```
END LOOP;  
CLOSE c_discos_canciones;  
END;  
/
```

Sin usar fetch

```
DECLARE  
CURSOR c_discos_canciones IS  
  SELECT d.nombre AS nombre_disco, COUNT(c.id) AS numero_canciones  
  FROM discos d  
  LEFT JOIN canciones c ON d.id = c.disco_id  
  GROUP BY d.nombre;  
BEGIN  
  FOR rec_disco IN c_discos_canciones LOOP  
    DBMS_OUTPUT.PUT_LINE('Disco: ' || rec_disco.nombre_disco || ', Canciones: ' ||  
rec_disco.numero_canciones);  
  END LOOP;  
END;  
/
```

PAQUETES

6. Crea un paquete de nombre discografica con para los procedimientos y funciones que crees en los siguientes ejercicios.

```
CREATE OR REPLACE PACKAGE pkg_discografica AS  
  PROCEDURE nombre_procedimiento;  
  --Declarar todos los procedimientos  
END pkg_discografica;  
/  
  
CREATE OR REPLACE PACKAGE BODY pkg_discografica AS  
  -- Implementación del procedimiento saludar  
  PROCEDURE nombre_procedimiento IS  
  BEGIN  
    --desarrollar código de procedimiento  
  END nombre_procedimiento;  
  
  --Implementar todos los procedimientos  
  
END pkg_discografica;  
/
```

PROCEDIMIENTOS

7. **Agregar una nueva canción a un disco**

Crea un procedimiento llamado agregar_cancion que reciba como parámetros el nombre de la canción, su duración y el ID del disco al que pertenece.

- Si el ID del disco no existe, debe generar un error controlado con un mensaje adecuado.
- Si la duración no es válida (por ejemplo, menor a 0 segundos), debe lanzar una excepción.
- Si la inserción es exitosa, debe confirmar la operación.

```
CREATE OR REPLACE PROCEDURE sp_agregar_cancion (  
  p_id_cancion IN canciones.id%TYPE,  
  p_nombre_cancion IN canciones.nombre%TYPE,  
  p_duracion IN canciones.duracion%TYPE,  
  p_disco_id IN canciones.disco_id%TYPE
```

```
) AS
v_disco_existe NUMBER;
ex_duracion_invalida EXCEPTION;
BEGIN
-- Verificar si el disco existe
SELECT COUNT(*) INTO v_disco_existe
FROM discos
WHERE id = p_disco_id;

IF v_disco_existe = 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'El ID del disco especificado no existe.');
```

END IF;

-- Verificar si la duración es válida (formato HH24:MI:SS)

IF NOT REGEXP_LIKE(p_duracion, '^([0-2][0-9]:[0-5][0-9]:[0-5][0-9])\$') THEN

RAISE ex_duracion_invalida;

END IF;

-- Insertar la canción

INSERT INTO canciones (id,nombre, duracion, disco_id)

VALUES (p_id_cancion,p_nombre_cancion, p_duracion, p_disco_id);

-- Confirmar la operación

COMMIT;

DBMS_OUTPUT.PUT_LINE('Canción "' || p_nombre_cancion || '" agregada exitosamente.');

EXCEPTION

WHEN ex_duracion_invalida THEN

ROLLBACK;

RAISE_APPLICATION_ERROR(-20002, 'La duración de la canción no es válida. Debe tener el formato HH24:MI:SS.');

WHEN OTHERS THEN

ROLLBACK;

RAISE_APPLICATION_ERROR(-20000, 'Ocurrió un error inesperado: ' || SQLERRM);

END;

/

--Bloque anónimo para probar el procedimiento

DECLARE

BEGIN

sp_agregar_cancion(500,'Al señor de la Oración','03:45:00',1);

END;

/

8. Registrar un nuevo integrante en un grupo

Desarrolla un procedimiento llamado `registrar_componente` que permita añadir un nuevo integrante a un grupo musical. Debe recibir el nombre del integrante, el instrumento que toca y el ID del grupo.

- Si el grupo no existe, debe lanzar una excepción con un mensaje de error.
- Si el integrante ya pertenece al grupo, debe evitar la inserción duplicada.
- Debe incluir control de errores para evitar fallos en la inserción.

```
CREATE OR REPLACE PROCEDURE sp_registrar_componente (
```

```
p_id_componente IN componentes.id%TYPE,
p_nombre_componente IN componentes.nombre%TYPE,
p_instrumento IN componentes.instrumento%TYPE,
p_grupo_id IN componentes.grupo_id%TYPE
) AS
v_grupo_existe NUMBER;
v_componente_existe NUMBER;
ex_grupo_inexistente EXCEPTION;
ex_componente_duplicado EXCEPTION;
BEGIN
-- Verificar si el grupo existe
SELECT COUNT(*) INTO v_grupo_existe
FROM grupos
WHERE id = p_grupo_id;

IF v_grupo_existe = 0 THEN
    RAISE ex_grupo_inexistente;
END IF;

-- Verificar si el componente ya existe en el grupo
SELECT COUNT(*) INTO v_componente_existe
FROM componentes
WHERE nombre = p_nombre_componente AND grupo_id = p_grupo_id;

IF v_componente_existe > 0 THEN
    RAISE ex_componente_duplicado;
END IF;

-- Insertar el componente
INSERT INTO componentes (id,nombre, instrumento, grupo_id)
VALUES (p_id_componente, p_nombre_componente, p_instrumento, p_grupo_id);

-- Confirmar la operación
COMMIT;
DBMS_OUTPUT.PUT_LINE('Componente ' || p_nombre_componente || ' agregado exitosamente al
grupo ' || p_grupo_id || '.');

EXCEPTION
WHEN ex_grupo_inexistente THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20001, 'El grupo con ID ' || p_grupo_id || ' no existe.');
```

```
WHEN ex_componente_duplicado THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20002, 'El componente ' || p_nombre_componente || ' ya existe en el
grupo ' || p_grupo_id || '.');
WHEN OTHERS THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20000, 'Ocurrió un error inesperado: ' || SQLERRM);
END;
/

declare

begin
sp_registrar_componente(100,'Nuevo Integrante', 'Batería', 1);
```



```
end;  
/
```

9. Actualizar la discografía de un grupo

Escribe un procedimiento llamado `cambiar_discografica` que permita actualizar la discografía de un grupo musical.

- Si el grupo no existe, debe lanzar una excepción con un mensaje adecuado.
- Si la discografía no existe en la tabla productoras, debe impedir la actualización.
- Si ocurre un error en la actualización, debe manejarlo y revertir cualquier cambio parcial.

```
CREATE OR REPLACE PROCEDURE sp_cambiar_discografica (  
    p_grupo_id IN grupos.id%TYPE,  
    p_discografica_id IN grupos.discografica_id%TYPE  
) AS  
    v_grupo_existe NUMBER;  
    v_discografica_existe NUMBER;  
    ex_grupo_inexistente EXCEPTION;  
    ex_discografica_inexistente EXCEPTION;  
BEGIN  
    -- Verificar si el grupo existe  
    SELECT COUNT(*) INTO v_grupo_existe  
    FROM grupos  
    WHERE id = p_grupo_id;  
  
    IF v_grupo_existe = 0 THEN  
        RAISE ex_grupo_inexistente;  
    END IF;  
  
    -- Verificar si la discografía existe  
    SELECT COUNT(*) INTO v_discografica_existe  
    FROM productoras  
    WHERE id = p_discografica_id;  
  
    IF v_discografica_existe = 0 THEN  
        RAISE ex_discografica_inexistente;  
    END IF;  
  
    -- Actualizar la discografía del grupo  
    UPDATE grupos  
    SET discografica_id = p_discografica_id  
    WHERE id = p_grupo_id;  
  
    -- Confirmar la operación  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Discografía del grupo ' || p_grupo_id || ' actualizada exitosamente.');
```

```
EXCEPTION  
    WHEN ex_grupo_inexistente THEN  
        ROLLBACK;  
        RAISE_APPLICATION_ERROR(-20001, 'El grupo con ID ' || p_grupo_id || ' no existe.');
```

```
    WHEN ex_discografica_inexistente THEN  
        ROLLBACK;  
        RAISE_APPLICATION_ERROR(-20002, 'La discografía con ID ' || p_discografica_id || ' no existe.');
```

```
    WHEN OTHERS THEN
```

```
ROLLBACK;
RAISE_APPLICATION_ERROR(-20000, 'Ocurrió un error inesperado: ' || SQLERRM);
END;
/
-- Actualización correcta
EXECUTE sp_cambiar_discografica(1, 2);
-- Grupo inexistente
EXECUTE sp_cambiar_discografica(99, 2);
-- Discográfica inexistente
EXECUTE sp_cambiar_discografica(1, 99);
/
```

10. Eliminar un disco y sus canciones

Crea un procedimiento eliminar_disco que reciba el ID de un disco y elimine tanto el disco como todas sus canciones asociadas.

- Si el disco no existe, debe lanzar una excepción con un mensaje informativo.
- Si ocurre un error durante la eliminación (por restricciones de clave foránea, por ejemplo), debe manejarlo sin afectar otros registros.
- Si la eliminación es exitosa, debe confirmar la operación.

```
CREATE OR REPLACE PROCEDURE sp_eliminar_disco (
    p_disco_id IN discos.id%TYPE
) AS
    v_disco_existe NUMBER;
    ex_disco_inexistente EXCEPTION;
BEGIN
    -- Verificar si el disco existe
    SELECT COUNT(*) INTO v_disco_existe
    FROM discos
    WHERE id = p_disco_id;

    IF v_disco_existe = 0 THEN
        RAISE ex_disco_inexistente;
    END IF;

    -- Eliminar las canciones asociadas al disco
    DELETE FROM canciones
    WHERE disco_id = p_disco_id;

    -- Eliminar el disco
    DELETE FROM discos
    WHERE id = p_disco_id;

    -- Confirmar la operación
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Disco con ID ' || p_disco_id || ' y sus canciones asociadas eliminados exitosamente.');
```

```
EXCEPTION
    WHEN ex_disco_inexistente THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'El disco con ID ' || p_disco_id || ' no existe.');
```

```
WHEN OTHERS THEN
    ROLLBACK;
```

```
RAISE_APPLICATION_ERROR(-20000, 'Ocurrió un error durante la eliminación: ' || SQLERRM);
END;
/
```

```
-- Eliminar un disco existente
EXECUTE sp_eliminar_disco(1);
-- Intentar eliminar un disco inexistente
EXECUTE sp_eliminar_disco(99);
```

11. Obtener el total de canciones de un disco

Diseña un procedimiento contar_canciones_disco que reciba el ID de un disco y devuelva la cantidad total de canciones que contiene. **El total de canciones debe ser un parámetro de salida.**

- Si el disco no existe, debe devolver un mensaje indicando que no se encontraron registros.
- Si ocurre algún error en la consulta, debe ser manejado y registrado adecuadamente.

```
CREATE OR REPLACE PROCEDURE sp_contar_canciones_disco (
    p_disco_id IN discos.id%TYPE,
    p_total_canciones OUT NUMBER
) AS
    v_disco_existe NUMBER;
    ex_disco_inexistente EXCEPTION;
BEGIN
    -- Verificar si el disco existe
    SELECT COUNT(*) INTO v_disco_existe
    FROM discos
    WHERE id = p_disco_id;

    IF v_disco_existe = 0 THEN
        RAISE ex_disco_inexistente;
    END IF;

    -- Contar las canciones del disco
    SELECT COUNT(*) INTO p_total_canciones
    FROM canciones
    WHERE disco_id = p_disco_id;

EXCEPTION
    WHEN ex_disco_inexistente THEN
        p_total_canciones := 0;
        DBMS_OUTPUT.PUT_LINE('No se encontraron registros para el disco con ID ' || p_disco_id || '.');
    WHEN OTHERS THEN
        p_total_canciones := -1; -- Valor de error
        DBMS_OUTPUT.PUT_LINE('Ocurrió un error al contar las canciones: ' || SQLERRM);
END;
/

--Bloque anónimo para probar el procedimiento
DECLARE
    v_total_canciones NUMBER;
BEGIN
    sp_contar_canciones_disco(2, v_total_canciones);
    DBMS_OUTPUT.PUT_LINE('Total canciones: ' || v_total_canciones);
    sp_contar_canciones_disco(99, v_total_canciones); -- Disco inexistente
    DBMS_OUTPUT.PUT_LINE('Total canciones: ' || v_total_canciones);
```

```
END;  
/
```

FUNCIONES

12. Obtener la duración total de un disco con control de errores

Crea una función `duracion_total_disco` que reciba el ID de un disco y devuelva la duración total de todas sus canciones.

- Si el disco no existe, la función debe devolver NULL y lanzar una excepción.
- Si el disco no tiene canciones, debe devolver 00:00:00.

```
CREATE OR REPLACE FUNCTION fn_duracion_total_disco (  
  p_disco_id IN discos.id%TYPE  
) RETURN VARCHAR2 AS  
  v_disco_existe NUMBER;  
  v_duracion_total_segundos NUMBER := 0;  
  ex_disco_inexistente EXCEPTION;  
  v_minutos NUMBER := 0;  
  v_segundos NUMBER := 0;  
  v_pos1 NUMBER;  
  v_pos2 NUMBER;  
  v_duracion_parte VARCHAR2(20);  
BEGIN  
  -- Verificar si el disco existe  
  SELECT COUNT(*) INTO v_disco_existe  
  FROM discos  
  WHERE id = p_disco_id;  
  
  IF v_disco_existe = 0 THEN  
    RAISE_APPLICATION_ERROR(-20001, 'No se encontró el disco con ID ' || p_disco_id || '.');  
  END IF;  
  
  -- Calcular la duración total de las canciones  
  FOR rec IN (SELECT duracion FROM canciones WHERE disco_id = p_disco_id) LOOP  
    -- Analizar la duración  
    v_pos1 := INSTR(rec.duracion, ':', 1, 1);  
    v_pos2 := INSTR(rec.duracion, ':', 1, 2);  
  
    IF v_pos2 > 0 THEN -- Formato HH:MM:SS  
      v_minutos := TO_NUMBER(SUBSTR(rec.duracion, 1, v_pos1 - 1));  
      v_segundos := TO_NUMBER(SUBSTR(rec.duracion, v_pos1 + 1, v_pos2 - v_pos1 - 1));  
      v_duracion_total_segundos := v_duracion_total_segundos + (v_minutos * 60) + v_segundos;  
    ELSE -- Formato MM:SS  
      v_segundos := TO_NUMBER(SUBSTR(rec.duracion, 1, v_pos1 - 1));  
      v_duracion_total_segundos := v_duracion_total_segundos + (v_minutos * 60);  
    END IF;  
  END LOOP;  
  
  -- Si no hay canciones, devolver 00:00:00  
  IF v_duracion_total_segundos = 0 THEN  
    RETURN '00:00:00';  
  ELSE  
    -- Convertir segundos totales a VARCHAR2 (HH24:MI:SS)  
    v_minutos := FLOOR(v_duracion_total_segundos / 60);  
    v_segundos := FLOOR(v_duracion_total_segundos - (v_minutos * 60));
```

```
RETURN LPAD(v_minutos, 2, '0') || ':' || LPAD(v_segundos, 2, '0');
END IF;

EXCEPTION
WHEN NO_DATA_FOUND THEN
RETURN '00:00:00'; -- Disco sin canciones
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Ocurrió un error al calcular la duración total: ' || SQLERRM);
RETURN NULL;
END;
/

--bloque anónimo de comprobación de la implementación
declare

begin
DBMS_OUTPUT.PUT_LINE(fn_duracion_total_disco(100));
end;
/
```

13. Obtener el nombre de la discográfica de un grupo

Crea una función nombre_discografica que reciba el nombre de un grupo y devuelva el nombre de su discográfica.

- Si el grupo no existe, debe devolver NULL y lanzar una excepción.
- Si el grupo no tiene discográfica asociada, debe devolver un mensaje indicando la situación.

```
CREATE OR REPLACE FUNCTION fn_nombre_discografica (
p_grupo_nombre IN grupos.nombre%TYPE
) RETURN VARCHAR2 AS
v_grupo_existe NUMBER;
v_discografica_id grupos.discografica_id%TYPE;
v_discografica_nombre productoras.nombre%TYPE;
ex_grupo_inexistente EXCEPTION;
BEGIN
-- Verificar si el grupo existe
SELECT COUNT(*) INTO v_grupo_existe
FROM grupos
WHERE nombre = p_grupo_nombre;

IF v_grupo_existe = 0 THEN
RAISE ex_grupo_inexistente;
END IF;

-- Obtener el ID de la discográfica del grupo
SELECT discografica_id INTO v_discografica_id
FROM grupos
WHERE id IN (SELECT id FROM GRUPOS WHERE nombre = p_grupo_nombre);

-- Verificar si el grupo tiene discográfica asociada
IF v_discografica_id IS NULL THEN
RETURN 'El grupo no tiene discográfica asociada.';
END IF;
```

```
-- Obtener el nombre de la discográfica
SELECT nombre INTO v_discografica_nombre
FROM productoras
WHERE id = v_discografica_id;

RETURN v_discografica_nombre;

EXCEPTION
WHEN ex_grupo_inexistente THEN
    RETURN NULL;
    DBMS_OUTPUT.PUT_LINE('No se encontró el grupo ' || p_grupo_nombre || ');
WHEN NO_DATA_FOUND THEN
    RETURN 'Discográfica no encontrada.';
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Ocurrió un error al obtener el nombre de la discográfica: ' || SQLERRM);
    RETURN NULL;
END;
/

--bloque anónimo de comprobación de la implementación
declare

begin
DBMS_OUTPUT.PUT_LINE(fn_nombre_discografica('The Beatles'));
end;
/
```

14. Contar cuántos integrantes tiene un grupo con validaciones

Crea una función cantidad_integrantes que reciba el ID de un grupo y devuelva la cantidad de integrantes que tiene.

- Si el grupo no existe, debe lanzar una excepción.
- Si el grupo no tiene integrantes, debe devolver 0.
- Debe manejar posibles errores de conexión o ejecución.

```
CREATE OR REPLACE FUNCTION sp_cantidad_integrantes (
    p_grupo_id IN grupos.id%TYPE
) RETURN NUMBER AS
    v_grupo_existe NUMBER;
    v_cantidad_integrantes NUMBER;
    ex_grupo_inexistente EXCEPTION;
BEGIN
    -- Verificar si el grupo existe
    SELECT COUNT(*) INTO v_grupo_existe
    FROM grupos
    WHERE id = p_grupo_id;

    IF v_grupo_existe = 0 THEN
        RAISE ex_grupo_inexistente;
    END IF;

    -- Contar los integrantes del grupo
    SELECT COUNT(*) INTO v_cantidad_integrantes
    FROM componentes
    WHERE grupo_id = p_grupo_id;
```

```
-- Si no hay integrantes, devolver 0
IF v_cantidad_integrantes IS NULL THEN
    RETURN 0;
ELSE
    RETURN v_cantidad_integrantes;
END IF;

EXCEPTION
WHEN ex_grupo_inexistente THEN
    RAISE_APPLICATION_ERROR(-20001, 'No se encontró el grupo con ID ' || p_grupo_id || ');
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Ocurrió un error al contar los integrantes: ' || SQLERRM);
    RETURN NULL;
END;
/

-- Bloque anónimo para comprobación
DECLARE
    v_cantidad_integrantes NUMBER;
BEGIN
    v_cantidad_integrantes := sp_cantidad_integrantes(1);
    DBMS_OUTPUT.PUT_LINE('Cantidad de integrantes: ' || v_cantidad_integrantes);

    v_cantidad_integrantes := sp_cantidad_integrantes(99); -- Grupo inexistente
    DBMS_OUTPUT.PUT_LINE('Cantidad de integrantes: ' || v_cantidad_integrantes);
END;
/
```

15. Determinar si una canción pertenece a un disco específico

Crea una función `es_cancion_de_disco` que reciba el nombre de una canción y el nombre de un disco, y devuelva TRUE o FALSE dependiendo de si la canción pertenece a ese disco.

- Si la canción o el disco no existen, debe lanzar una excepción.
- Si la canción no pertenece al disco, debe devolver FALSE.
- Manejar errores inesperados en la ejecución.

```
CREATE OR REPLACE FUNCTION fn_es_cancion_de_disco (
    p_nombre_cancion IN canciones.nombre%TYPE,
    p_nombre_disco IN discos.nombre%TYPE
) RETURN BOOLEAN AS
    v_cancion_existe NUMBER;
    v_disco_existe NUMBER;
    v_cancion_disco_id canciones.disco_id%TYPE;
    v_disco_id discos.id%TYPE;
    v_cancion_id canciones.id%TYPE;
    ex_cancion_inexistente EXCEPTION;
    ex_disco_inexistente EXCEPTION;
BEGIN
    -- Verificar si el disco existe
    SELECT COUNT(*) INTO v_disco_existe
    FROM discos
    WHERE nombre = p_nombre_disco;

    IF v_disco_existe = 0 THEN
```

```
    RAISE ex_disco_inexistente;
END IF;

-- Verificar si la canción existe
SELECT COUNT(*) INTO v_cancion_existe
FROM canciones
WHERE nombre = p_nombre_cancion;

IF v_cancion_existe = 0 THEN
    RAISE ex_cancion_inexistente;
END IF;

-- Obtener el ID del disco
SELECT id INTO v_disco_id
FROM discos
WHERE nombre = p_nombre_disco;

-- Obtener el ID de la canción
SELECT id INTO v_cancion_id
FROM canciones
WHERE nombre = p_nombre_cancion;

-- Obtener el ID del disco de la canción
SELECT disco_id INTO v_cancion_disco_id
FROM canciones
WHERE id = v_cancion_id;

-- Verificar si la canción pertenece al disco
IF v_cancion_disco_id = v_disco_id THEN
    RETURN TRUE;
ELSE
    RETURN FALSE;
END IF;

EXCEPTION
    WHEN ex_cancion_inexistente THEN
        RAISE_APPLICATION_ERROR(-20001, 'No se encontró la canción con nombre ' || p_nombre_cancion ||
        '.');
    WHEN ex_disco_inexistente THEN
        RAISE_APPLICATION_ERROR(-20002, 'No se encontró el disco con nombre ' || p_nombre_disco || '.');
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE; -- Canción no encontrada en el disco
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Ocurrió un error inesperado: ' || SQLERRM);
        RETURN NULL;
END;
/

DECLARE
    v_resultado BOOLEAN;
BEGIN
    -- Caso 1: Canción y disco existen, canción pertenece al disco
    v_resultado := fn_es_cancion_de_disco('Bad', 'The Unforgettable Fire');
    DBMS_OUTPUT.PUT_LINE('Caso 1: ' || CASE WHEN v_resultado THEN 'TRUE' ELSE 'FALSE' END);
```



```
-- Caso 2: Canción y disco existen, canción no pertenece al disco
v_resultado := fn_es_cancion_de_disco('42', 'The Unforgettable Fire');
DBMS_OUTPUT.PUT_LINE('Caso 2: ' || CASE WHEN v_resultado THEN 'TRUE' ELSE 'FALSE' END);

-- Caso 3: Canción existe, disco no existe
BEGIN
  v_resultado := fn_es_cancion_de_disco('42', 'V Estación');
  DBMS_OUTPUT.PUT_LINE('Caso 3: ' || CASE WHEN v_resultado THEN 'TRUE' ELSE 'FALSE' END);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Caso 3: Excepción capturada');
END;

-- Caso 4: Canción no existe, disco existe
BEGIN
  v_resultado := fn_es_cancion_de_disco('Al Gitano de la Cava', 'The Unforgettable Fire');
  DBMS_OUTPUT.PUT_LINE('Caso 4: ' || CASE WHEN v_resultado THEN 'TRUE' ELSE 'FALSE' END);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Caso 4: Excepción capturada');
END;

-- Caso 5: Canción y disco no existen
BEGIN
  v_resultado := fn_es_cancion_de_disco('Al Gitano de la Cava', 'Eternidad');
  DBMS_OUTPUT.PUT_LINE('Caso 5: ' || CASE WHEN v_resultado THEN 'TRUE' ELSE 'FALSE' END);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Caso 5: Excepción capturada');
END;
END;
/
```

16. Obtener el nombre del grupo al que pertenece un integrante

Crea una función grupo_de_integrante que reciba el nombre de un integrante y devuelva el nombre del grupo al que pertenece.

- Si el integrante no existe, debe lanzar una excepción.
- Si el integrante no está asignado a un grupo, debe devolver un mensaje indicando que no pertenece a ningún grupo.
- Manejar posibles errores en la consulta.

```
CREATE OR REPLACE FUNCTION fn_grupo_de_integrante (
  p_nombre_integrante IN componentes.nombre%TYPE
) RETURN VARCHAR2 AS
  v_componente_existe NUMBER;
  v_grupo_id componentes.grupo_id%TYPE;
  v_grupo_nombre grupos.nombre%TYPE;
  ex_componente_inexistente EXCEPTION;
BEGIN
  -- Verificar si el integrante existe
  SELECT COUNT(*) INTO v_componente_existe
  FROM componentes
  WHERE nombre = p_nombre_integrante;
```

```
IF v_componente_existe = 0 THEN
    RAISE ex_componente_inexistente;
END IF;

-- Obtener el ID del grupo del integrante
SELECT grupo_id INTO v_grupo_id
FROM componentes
WHERE nombre = p_nombre_integrante;

-- Verificar si el integrante está asignado a un grupo
IF v_grupo_id IS NULL THEN
    RETURN 'El integrante no pertenece a ningún grupo.';
END IF;

-- Obtener el nombre del grupo
SELECT nombre INTO v_grupo_nombre
FROM grupos
WHERE id = v_grupo_id;

RETURN v_grupo_nombre;

EXCEPTION
    WHEN ex_componente_inexistente THEN
        RAISE_APPLICATION_ERROR(-20001, 'No se encontró el integrante con nombre ' ||
p_nombre_integrante || '.');
    WHEN NO_DATA_FOUND THEN
        RETURN 'Grupo no encontrado.';
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Ocurrió un error al obtener el nombre del grupo: ' || SQLERRM);
        RETURN NULL;
END;
/

DECLARE
    v_grupo_nombre VARCHAR2(255);
BEGIN
    -- Caso 1: Integrante existe y pertenece a un grupo
    v_grupo_nombre := fn_grupo_de_integrante('Freddie Mercury');
    DBMS_OUTPUT.PUT_LINE('Caso 1: Grupo - ' || v_grupo_nombre);

    -- Caso 2: Integrante existe y no pertenece a ningún grupo
    v_grupo_nombre := fn_grupo_de_integrante('Nuevo Integrante');
    DBMS_OUTPUT.PUT_LINE('Caso 2: ' || v_grupo_nombre);

    -- Caso 3: Integrante no existe
    BEGIN
        v_grupo_nombre := fn_grupo_de_integrante('Ana Torroja');
        DBMS_OUTPUT.PUT_LINE('Caso 3: Grupo - ' || v_grupo_nombre);
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Caso 3: Excepción capturada - Integrante no existe');
    END;

    -- Caso 4: Error inesperado (simulado)
    BEGIN
```

```
-- Simulación de un error (por ejemplo, una conversión inválida)
v_grupo_nombre := TO_NUMBER('abc');
DBMS_OUTPUT.PUT_LINE('Caso 4: Grupo - ' || v_grupo_nombre);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Caso 4: Excepción capturada - Error inesperado');
END;
END;
/
```

TRIGGERS

17. Evitar la eliminación de discos si tienen canciones asociadas

Crea un **trigger BEFORE DELETE** en la tabla discos que impida eliminar un disco si aún tiene canciones registradas.

- Si hay canciones asociadas, debe lanzar un error con un mensaje adecuado.
- Si el disco no tiene canciones, la eliminación se permite.

```
CREATE OR REPLACE TRIGGER trg_evitar_eliminar_disco
BEFORE DELETE ON discos
FOR EACH ROW
DECLARE
  v_cantidad_canciones NUMBER;
BEGIN
  -- Contar las canciones asociadas al disco
  SELECT COUNT(*) INTO v_cantidad_canciones
  FROM canciones
  WHERE disco_id = :OLD.id;

  -- Si hay canciones asociadas, lanzar una excepción
  IF v_cantidad_canciones > 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'No se puede eliminar el disco porque aún tiene canciones registradas.');
```

18. Registrar automáticamente la fecha de publicación de un disco

Crea un **trigger BEFORE INSERT** en la tabla discos que asigne automáticamente la fecha actual (SYSDATE) a la columna fecha_publicacion si el usuario no proporciona una.

```
CREATE OR REPLACE TRIGGER trg_asignar_fecha_publicacion
BEFORE INSERT ON discos
FOR EACH ROW
BEGIN
  -- Si la fecha de publicación no se proporciona, asignar SYSDATE
  IF :NEW.fecha_publicacion IS NULL THEN
```

```
:NEW.fecha_publicacion := SYSDATE;
END IF;
END;
/

declare

begin
    INSERT INTO discos VALUES (500,'Costalero del Soberano',null,2);
end;
/
```

19. Registrar cambios en la discográfica de un grupo

Crea un **trigger AFTER UPDATE** en la tabla grupos que registre en una tabla de auditoría (historial_discograficas) cada vez que se cambie la discográfica de un grupo.

- Debe almacenar el ID del grupo, la discográfica anterior, la nueva discográfica y la fecha del cambio.
- Si ocurre un error en el registro del historial, debe manejarse sin afectar la actualización original.

```
-- Creación de la tabla historial_discograficas
declare

begin
    EXECUTE IMMEDIATE 'CREATE TABLE historial_discograficas (
        grupo_id NUMBER,
        discografica_anterior NUMBER,
        discografica_nueva NUMBER,
        fecha_cambio DATE
    )
    ';
end;
/

--Trigger
CREATE OR REPLACE TRIGGER trg_registrar_cambio_discografica
AFTER UPDATE OF discografica_id ON grupos
FOR EACH ROW
BEGIN
    -- Verificar si la discográfica ha cambiado
    IF :OLD.discografica_id <> :NEW.discografica_id THEN
        BEGIN
            -- Insertar en la tabla de auditoría
            INSERT INTO historial_discograficas (grupo_id, discografica_anterior, discografica_nueva,
            fecha_cambio)
            VALUES (:OLD.id, :OLD.discografica_id, :NEW.discografica_id, SYSDATE);
        EXCEPTION
            WHEN OTHERS THEN
                -- Manejar errores sin afectar la actualización original
                DBMS_OUTPUT.PUT_LINE('Error al registrar el cambio de discográfica: ' || SQLERRM);
        END;
    END IF;
END;
/
```

```
--Bloque anónimo para probar el trigger  
declare
```

```
begin  
UPDATE musica_grupos SET discografica_id = 2 WHERE id = 1;  
end;  
/
```

20. Evitar nombres duplicados en las canciones de un mismo disco

Crea un **trigger BEFORE INSERT OR UPDATE** en la tabla canciones que verifique que no haya otra canción con el mismo nombre en el mismo disco.

- Si ya existe una canción con el mismo nombre dentro del disco, debe lanzar un error e impedir la inserción o actualización.

```
CREATE OR REPLACE TRIGGER trg_cancion_unica_por_disco  
BEFORE INSERT OR UPDATE ON canciones  
FOR EACH ROW  
DECLARE  
    v_canciones_existentes NUMBER;  
BEGIN  
    -- Verificar si existe otra canción con el mismo nombre en el mismo disco  
    SELECT COUNT(*) INTO v_canciones_existentes  
    FROM canciones  
    WHERE nombre = :NEW.nombre  
    AND disco_id = :NEW.disco_id  
    AND id <> :NEW.id; -- Excluir la canción actual en caso de actualización  
  
    -- Si existe otra canción con el mismo nombre, lanzar una excepción  
    IF v_canciones_existentes > 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Ya existe una canción con el mismo nombre en este disco.');    END IF;  
END;  
/  
  
--bloque anónimo para comprobación del trigger  
declare  
  
begin  
    INSERT INTO canciones VALUES (100, 'Come Together', '00:00:00', 0);  
end;  
/
```

21. Actualizar automáticamente la nacionalidad de una discográfica

Crea un **trigger AFTER INSERT** en la tabla productoras que asigne automáticamente la nacionalidad "Desconocida" si el usuario no proporciona un valor en la columna nacionalidad.

```
CREATE OR REPLACE TRIGGER trg_asignar_nacionalidad_desconocida  
AFTER INSERT ON productoras  
FOR EACH ROW  
BEGIN  
    -- Si la nacionalidad no se proporciona, asignar "Desconocida"  
    IF :NEW.nacionalidad IS NULL THEN  
        UPDATE productoras
```

```
SET nacionalidad = 'Desconocida'
WHERE id = :NEW.id;
END IF;
END;
/

--Bloque anónimo de comprobación del trigger
declare

begin
    INSERT INTO productoras (id,nombre) VALUES (500,'Málaga Cofradiera Music');
end;
/
```

